

ITEC623 - Deep Learning and Emerging Topics in Artificial Intelligence

Lab 4: Convolutional Neural Networks: Architecture, Parameters & Tuning

Student Name: Emmanuel Orunta **Student ID:** S00403657

Lab Overview

This lab focuses on designing, training and tuning a Convolutional Neural Network (CNN) for image classification using TensorFlow/Keras. The MNIST handwritten-digit dataset is loaded with a reproducible setup and passed through a tf.data input pipeline. A baseline CNN is then built to match a given reference architecture, verified with model.summary(), trained and evaluated with error analysis. Finally, one design parameter is changed to observe its effect on model size, accuracy and compute, and the lab ends with a reflection on running and deploying the model in Azure.

In [1]:

```
# Importing Libraries

import os
import random
import time
import platform

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf

from tensorflow import keras
from tensorflow.keras import layers, models
from sklearn.metrics import confusion_matrix, classification_report

print("Python version:", platform.python_version())
print("TensorFlow version:", tf.__version__)
print("NumPy version:", np.__version__)
```

```
Python version: 3.13.15
TensorFlow version: 2.20.0
NumPy version: 2.1.3
```

TASK 1 — Reproducible Setup and Dataset Preparation

In [2]:

```
# Reproducibility setup

SEED = 42

os.environ["PYTHONHASHSEED"] = str(SEED)

random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)
keras.utils.set_random_seed(SEED)

# Make TensorFlow operations deterministic (same results on every run)
tf.config.experimental.enable_op_determinism()

print("Python random seed:", SEED)
print("NumPy random seed:", SEED)
print("TensorFlow random seed:", SEED)
print("TensorFlow op determinism: enabled")
```

```
Python random seed: 42
NumPy random seed: 42
TensorFlow random seed: 42
TensorFlow op determinism: enabled
```

In [3]:

```
# Checking the hardware available

gpus = tf.config.list_physical_devices("GPU")

if gpus:
    print("GPU available:", gpus)
else:
    print("No GPU found - training will run on the CPU.")
```

```
GPU available: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

In [4]:

```
# Load MNIST

(X_train_full, y_train_full), (X_test, y_test) = keras.datasets.mnist.load_data()

print("Training images shape:", X_train_full.shape)
print("Training labels shape:", y_train_full.shape)

print("Test images shape:", X_test.shape)
print("Test labels shape:", y_test.shape)
```

```
Training images shape: (60000, 28, 28)
Training labels shape: (60000,)
Test images shape: (10000, 28, 28)
Test labels shape: (10000,)
```

In [5]:

```
# Creating a validation set from the training data

VAL_SIZE = 6000

rng = np.random.default_rng(SEED)
shuffled_indices = rng.permutation(len(X_train_full))

val_indices = shuffled_indices[:VAL_SIZE]
train_indices = shuffled_indices[VAL_SIZE:]

X_train, y_train = X_train_full[train_indices], y_train_full[train_indices]
X_val, y_val = X_train_full[val_indices], y_train_full[val_indices]

print("Training set:", X_train.shape, y_train.shape)
print("Validation set:", X_val.shape, y_val.shape)
print("Test set:", X_test.shape, y_test.shape)
```

```
Training set: (54000, 28, 28) (54000,)
Validation set: (6000, 28, 28) (6000,)
Test set: (10000, 28, 28) (10000,)
```

In [6]:

```
# Display example labels

print("First 10 numerical labels:")
print(y_train[:10])
```

```
First 10 numerical labels:
[0 9 2 3 8 0 4 5 5 6]
```

In [7]:

```
# Visualising ten MNIST samples

plt.figure(figsize=(12, 3))

for i in range(10):
    plt.subplot(1, 10, i + 1)
    plt.imshow(X_train[i], cmap="gray")
    plt.title(str(y_train[i]))
    plt.axis("off")

plt.tight_layout()
plt.show()
```



In [8]:

```
# Label distribution

distribution_table = pd.DataFrame({
    "Digit": range(10),
    "Train": np.bincount(y_train, minlength=10),
    "Validation": np.bincount(y_val, minlength=10),
    "Test": np.bincount(y_test, minlength=10)
})

distribution_table["Train %"] = (
    distribution_table["Train"] / distribution_table["Train"].sum() * 100
).round(2)

distribution_table
```

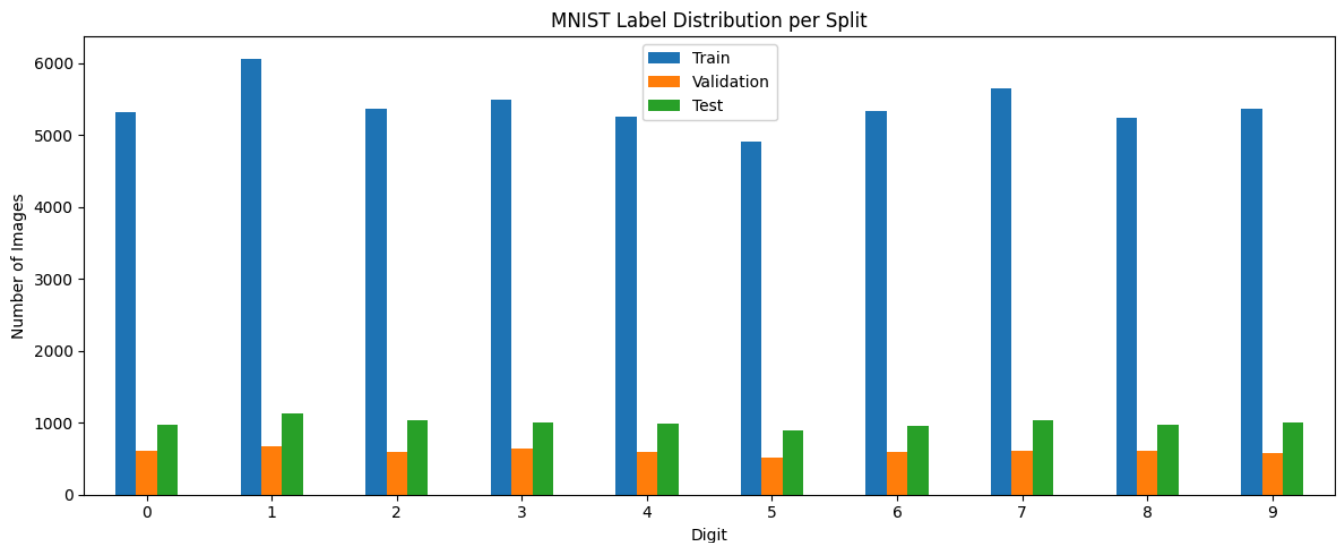
	Digit	Train	Validation	Test	Train %
0	0	5319	604	980	9.85
1	1	6067	675	1135	11.24
2	2	5361	597	1032	9.93
3	3	5487	644	1010	10.16
4	4	5255	587	982	9.73
5	5	4913	508	892	9.10
6	6	5329	589	958	9.87
7	7	5657	608	1028	10.48
8	8	5243	608	974	9.71
9	9	5369	580	1009	9.94

In [9]:

```
# Plot label distribution

distribution_table.plot(
    x="Digit",
    y=["Train", "Validation", "Test"],
    kind="bar",
    figsize=(12, 5),
    rot=0
)

plt.xlabel("Digit")
plt.ylabel("Number of Images")
plt.title("MNIST Label Distribution per Split")
plt.tight_layout()
plt.show()
```



I used the MNIST dataset, which contains 70,000 greyscale images of handwritten digits from 0 to 9. Each image is 28×28 pixels with a single colour channel. The dataset provides 60,000 training images and 10,000 test images. I kept the official test set separate and used a seeded random split to hold out 6,000 of the training images as a validation set, leaving 54,000 images for training. The label distribution shows that each digit makes up between 9.10% (digit 5) and 11.24% (digit 1) of the training data, so the classes are close to balanced and accuracy is a fair measure of performance. The notebook was run with Python 3.13.15, TensorFlow 2.20.0 and NumPy 2.1.3 on a Google Colab GPU.

I set the same random seed, 42, for Python, NumPy and TensorFlow, and enabled TensorFlow op determinism. Reproducibility matters because weight initialisation, data shuffling and dropout are all random, so without fixed seeds two runs of the same code can give slightly different accuracies. With a fixed setup, any change in results can be linked to a change I made to the model rather than to chance, and other people using the same library versions and hardware can rerun the notebook and get the same numbers.

TASK 2 — Input Pipeline with tf.data

In [10]:

```
# Inspecting raw pixel values

print("Raw data type:", X_train.dtype)
print("Raw minimum pixel value:", X_train.min())
print("Raw maximum pixel value:", X_train.max())
```

```
Raw data type: uint8
Raw minimum pixel value: 0
Raw maximum pixel value: 255
```

In [11]:

```
# Preprocessing function for the pipeline

def preprocess_image(image, label):
    image = tf.cast(image, tf.float32) / 255.0 # normalise to [0, 1]
    image = tf.expand_dims(image, axis=-1)      # (28, 28) -> (28, 28, 1)
    return image, label
```

In [12]:

```
# Build training pipeline

BATCH_SIZE = 64

train_pipeline = (
    tf.data.Dataset.from_tensor_slices((X_train, y_train))
    .map(
        preprocess_image,
        num_parallel_calls=tf.data.AUTOTUNE
    )
    .cache()
    .shuffle(
        buffer_size=len(X_train),
        seed=SEED,
        reshuffle_each_iteration=True
    )
    .batch(BATCH_SIZE)
    .prefetch(tf.data.AUTOTUNE)
)

print("Training pipeline created.")
```

Training pipeline created.

In [13]:

```
# Build validation and test pipelines (no shuffling)

val_pipeline = (
    tf.data.Dataset.from_tensor_slices((X_val, y_val))
    .map(preprocess_image, num_parallel_calls=tf.data.AUTOTUNE)
    .cache()
    .batch(BATCH_SIZE)
    .prefetch(tf.data.AUTOTUNE)
)

test_pipeline = (
    tf.data.Dataset.from_tensor_slices((X_test, y_test))
    .map(preprocess_image, num_parallel_calls=tf.data.AUTOTUNE)
    .cache()
    .batch(BATCH_SIZE)
    .prefetch(tf.data.AUTOTUNE)
)

print("Validation pipeline created.")
print("Test pipeline created.")

print("\nNumber of training batches:", train_pipeline.cardinality().numpy())
print("Number of validation batches:", val_pipeline.cardinality().numpy())
print("Number of test batches:", test_pipeline.cardinality().numpy())
```

Validation pipeline created.
Test pipeline created.

Number of training batches: 844
Number of validation batches: 94
Number of test batches: 157

In [14]:

```
# Inspect one training batch

for batch_images, batch_labels in train_pipeline.take(1):

    print("Batch image shape:", batch_images.shape)
    print("Batch label shape:", batch_labels.shape)

    print("Batch data type:", batch_images.dtype)

    print("Batch minimum value:", tf.reduce_min(batch_images).numpy())
    print("Batch maximum value:", tf.reduce_max(batch_images).numpy())
```

```
Batch image shape: (64, 28, 28, 1)
Batch label shape: (64,)
Batch data type: <dtype: 'float32'>
Batch minimum value: 0.0
Batch maximum value: 1.0
```

In [15]:

```
# Visualise images coming directly from the tf.data pipeline

for batch_images, batch_labels in train_pipeline.take(1):

    plt.figure(figsize=(12, 3))

    for i in range(10):
        plt.subplot(1, 10, i + 1)
        plt.imshow(batch_images[i, :, :, 0].numpy(), cmap="gray")
        plt.title(str(batch_labels[i].numpy()))
        plt.axis("off")

    plt.tight_layout()
    plt.show()
```



In [16]:

```
# Comparing pipeline speed (basic vs optimised)

def time_one_pass(dataset):
    start = time.perf_counter()
    for _ in dataset:
        pass
    return time.perf_counter() - start

basic_pipeline = (
    tf.data.Dataset.from_tensor_slices((X_train, y_train))
    .map(preprocess_image)
    .batch(BATCH_SIZE)
)

time_one_pass(train_pipeline) # first pass fills the cache

print(f"Basic pipeline (no parallel map, cache or prefetch): {time_one_pass(basic_pipeline):.2f} seconds")
print(f"Optimised pipeline (parallel map + cache + prefetch): {time_one_pass(train_pipeline):.2f} seconds")
```

Basic pipeline (no parallel map, cache or prefetch): 1.67 seconds
Optimised pipeline (parallel map + cache + prefetch): 0.32 seconds

Pipeline Workflow

The data pipeline follows the sequence:

Raw images → map / normalise and add channel → cache → shuffle → batch → prefetch → model

The validation and test pipelines use the same steps except shuffling, so evaluation is always done in the same order.

The raw MNIST images are stored as uint8 values between 0 and 255. In the mapping stage I converted them to float32, divided by 255 so that every pixel is between 0 and 1, and added a channel dimension because Conv2D layers expect input in the form (height, width, channels). The printed batch shape of (64, 28, 28, 1) with a minimum of 0.0 and a maximum of 1.0 confirms that the pipeline is working correctly, and the 54,000 training images are split into 844 batches. Caching stores the processed images in memory after the first epoch, shuffling with a seed changes the order each epoch in a reproducible way, batching groups 64 images for mini-batch training, and prefetching prepares the next batch while the model is training on the current one. In the timing comparison, one pass through the training data took 0.32 seconds with the optimised pipeline compared with 1.67 seconds for the basic version, about five times faster, which reduces the time the GPU spends waiting for data.

TASK 3 — Baseline CNN Build (Must Match Reference Summary)

In [17]:

```
# Building the baseline CNN

def build_cnn(conv2_filters=64, model_name="baseline_cnn"):
    model = models.Sequential([
        layers.Input(shape=(28, 28, 1)),
        layers.Conv2D(32, kernel_size=(3, 3), padding="same", activation="relu", name="conv1"),
        layers.Conv2D(conv2_filters, kernel_size=(3, 3), padding="same", activation="relu",
name="conv2"),
        layers.MaxPooling2D(pool_size=(2, 2), name="maxpool"),
        layers.Flatten(name="flatten"),
        layers.Dropout(0.25, name="dropout1"),
        layers.Dense(128, activation="relu", name="dense1"),
        layers.Dropout(0.5, name="dropout2"),
        layers.Dense(10, activation="softmax", name="output")
    ], name=model_name)
    return model

keras.utils.set_random_seed(SEED)
baseline_model = build_cnn()

print("Baseline CNN built.")
```

Baseline CNN built.

In [18]:

```
# Baseline model summary

baseline_model.summary()
```

Model: "baseline_cnn"

Layer (type)	Output Shape	Param #
conv1 (Conv2D)	(None, 28, 28, 32)	320

conv2 (Conv2D)	(None, 28, 28, 64)	18,496
maxpool (MaxPooling2D)	(None, 14, 14, 64)	0
flatten (Flatten)	(None, 12544)	0
dropout1 (Dropout)	(None, 12544)	0
dense1 (Dense)	(None, 128)	1,605,760
dropout2 (Dropout)	(None, 128)	0
output (Dense)	(None, 10)	1,290

Total params: 1,625,866 (6.20 MB)

Trainable params: 1,625,866 (6.20 MB)

Non-trainable params: 0 (0.00 B)

In [19]:

```
# Checking the model against the reference architecture
```

```
reference = [
    ("conv1",    (None, 28, 28, 32), 320),
    ("conv2",    (None, 28, 28, 64), 18496),
    ("maxpool",  (None, 14, 14, 64), 0),
    ("flatten",  (None, 12544),      0),
    ("dropout1", (None, 12544),      0),
    ("dense1",   (None, 128),        1605760),
    ("dropout2", (None, 128),        0),
    ("output",   (None, 10),         1290),
]

rows = []

for layer_name, expected_shape, expected_params in reference:
    layer = baseline_model.get_layer(layer_name)
    actual_shape = tuple(layer.output.shape)
    actual_params = layer.count_params()
    match = "Yes" if (actual_shape == expected_shape and actual_params == expected_params) else
    "No"
    rows.append([layer_name, actual_shape, expected_shape, actual_params, expected_params, match])

check_table = pd.DataFrame(
    rows,
    columns=["Layer", "Output Shape", "Expected Shape", "Params", "Expected Params", "Match"]
)

print("Total parameters:", baseline_model.count_params())
print("All layers match the reference:", (check_table["Match"] == "Yes").all())

check_table
```

Total parameters: 1625866

All layers match the reference: True

	Layer	Output Shape	Expected Shape	Params	Expected Params	Match
0	conv1	(None, 28, 28, 32)	(None, 28, 28, 32)	320	320	Yes
1	conv2	(None, 28, 28, 64)	(None, 28, 28, 64)	18496	18496	Yes
2	maxpool	(None, 14, 14, 64)	(None, 14, 14, 64)	0	0	Yes
3	flatten	(None, 12544)	(None, 12544)	0	0	Yes
4	dropout1	(None, 12544)	(None, 12544)	0	0	Yes
5	dense1	(None, 128)	(None, 128)	1605760	1605760	Yes
6	dropout2	(None, 128)	(None, 128)	0	0	Yes
7	output	(None, 10)	(None, 10)	1290	1290	Yes

How the Parameter Counts Are Calculated

Layer	Calculation	Parameters
Conv2D (32)	$(3 \times 3 \times 1 \text{ input channel}) \times 32 \text{ filters} + 32 \text{ biases}$	320
Conv2D (64)	$(3 \times 3 \times 32 \text{ input channels}) \times 64 \text{ filters} + 64 \text{ biases}$	18,496
MaxPooling2D	no weights, halves 28×28 to 14×14	0
Flatten	$14 \times 14 \times 64 = 12,544$ values, no weights	0
Dropout	no weights	0
Dense (128)	$12,544 \times 128 + 128 \text{ biases}$	1,605,760
Dropout	no weights	0
Dense (10)	$128 \times 10 + 10 \text{ biases}$	1,290
Total		1,625,866

I built the baseline CNN with two 3×3 convolution layers (32 and 64 filters) using `padding='same'`, followed by max pooling, flattening, dropout, a dense layer of 128 neurons, a second dropout layer and a 10-class softmax output. The model summary and the check table show that every output shape and parameter count matches the reference, with a total of 1,625,866 parameters. A convolution layer's parameters depend only on the kernel size, the number of input channels and the number of filters, plus one bias per filter, which is why the first convolution has only 320 parameters and the second has 18,496. `Padding='same'` keeps the image at 28×28 after each convolution, and the pooling layer halves it to 14×14 . The first dense layer holds about 98.8% of all parameters because it connects all 12,544 flattened values to 128 neurons, so the size of the feature map entering Flatten is the main driver of model size.

TASK 4 — Training, Evaluation and Error Analysis

In [20]:

```
# Callback to record the time taken for each epoch

class EpochTimer(keras.callbacks.Callback):
    def on_train_begin(self, logs=None):
        self.times = []

    def on_epoch_begin(self, epoch, logs=None):
        self.start = time.perf_counter()

    def on_epoch_end(self, epoch, logs=None):
        self.times.append(time.perf_counter() - self.start)
```

In [21]:

```
# Training settings

EPOCHS = 10

print("Optimiser: Adam (learning rate 0.001)")
print("Loss: sparse categorical cross-entropy")
print("Epochs:", EPOCHS)
```

```
Optimiser: Adam (learning rate 0.001)
Loss: sparse categorical cross-entropy
Epochs: 10
```

In [22]:

```
# Train the baseline CNN
# The model is rebuilt here so that re-running this cell always starts from fresh
# weights (otherwise Keras would continue training the already-trained model).
```

```
keras.utils.set_random_seed(SEED)
baseline_model = build_cnn()

baseline_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
baseline_timer = EpochTimer()
```

```
baseline_history = baseline_model.fit(
    train_pipeline,
    validation_data=val_pipeline,
    epochs=EPOCHS,
    callbacks=[baseline_timer],
    verbose=2
)
```

```
Epoch 1/10
844/844 - 14s - 16ms/step - accuracy: 0.9377 - loss: 0.2048 - val_accuracy: 0.9810 - val_loss: 0.0626
Epoch 2/10
844/844 - 8s - 9ms/step - accuracy: 0.9764 - loss: 0.0794 - val_accuracy: 0.9863 - val_loss: 0.0534
Epoch 3/10
844/844 - 8s - 9ms/step - accuracy: 0.9817 - loss: 0.0598 - val_accuracy: 0.9890 - val_loss: 0.0426
Epoch 4/10
844/844 - 7s - 9ms/step - accuracy: 0.9841 - loss: 0.0509 - val_accuracy: 0.9893 - val_loss: 0.0375
Epoch 5/10
844/844 - 8s - 9ms/step - accuracy: 0.9872 - loss: 0.0405 - val_accuracy: 0.9887 - val_loss: 0.0400
Epoch 6/10
844/844 - 10s - 12ms/step - accuracy: 0.9883 - loss: 0.0366 - val_accuracy: 0.9888 - val_loss: 0.0434
Epoch 7/10
844/844 - 7s - 9ms/step - accuracy: 0.9895 - loss: 0.0315 - val_accuracy: 0.9885 - val_loss: 0.0438
Epoch 8/10
844/844 - 8s - 9ms/step - accuracy: 0.9904 - loss: 0.0287 - val_accuracy: 0.9890 - val_loss: 0.0437
Epoch 9/10
844/844 - 7s - 8ms/step - accuracy: 0.9913 - loss: 0.0256 - val_accuracy: 0.9900 - val_loss: 0.0403
Epoch 10/10
844/844 - 10s - 11ms/step - accuracy: 0.9921 - loss: 0.0239 - val_accuracy: 0.9912 - val_loss: 0.0396
```

In [23]:

```
# Training history table
```

```
baseline_history_table = pd.DataFrame(baseline_history.history).round(4)
baseline_history_table.index = baseline_history_table.index + 1
baseline_history_table.index.name = "Epoch"
```

```
baseline_history_table
```

	accuracy	loss	val_accuracy	val_loss
Epoch				
1	0.9377	0.2048	0.9810	0.0626
2	0.9764	0.0794	0.9863	0.0534
3	0.9817	0.0598	0.9890	0.0426
4	0.9841	0.0509	0.9893	0.0375
5	0.9872	0.0405	0.9887	0.0400
6	0.9883	0.0366	0.9888	0.0434
7	0.9895	0.0315	0.9885	0.0438
8	0.9904	0.0287	0.9890	0.0437
9	0.9913	0.0256	0.9900	0.0403
10	0.9921	0.0239	0.9912	0.0396

In [24]:

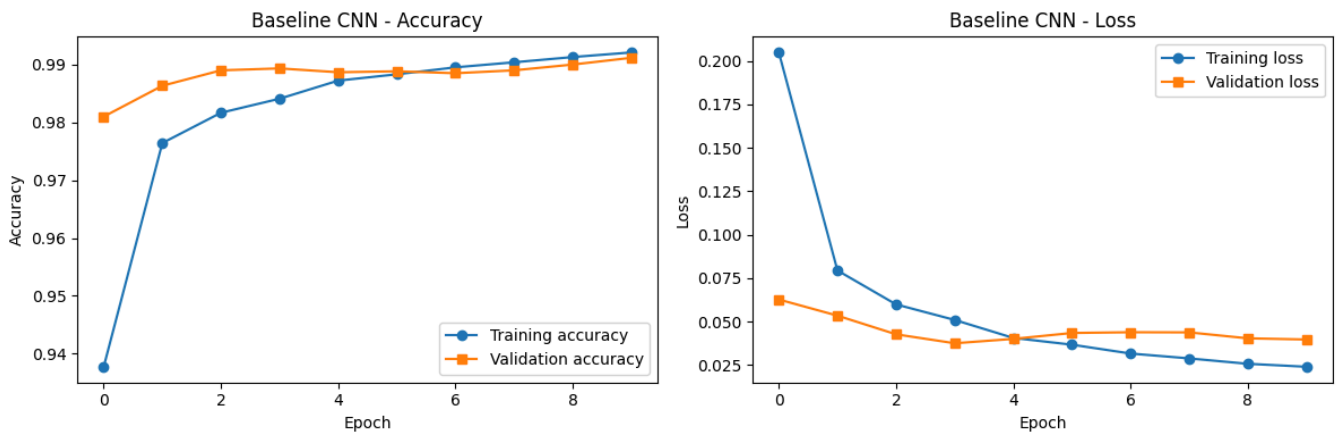
```
# Plot accuracy and loss curves

plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(baseline_history.history["accuracy"], marker="o", label="Training accuracy")
plt.plot(baseline_history.history["val_accuracy"], marker="s", label="Validation accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Baseline CNN - Accuracy")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(baseline_history.history["loss"], marker="o", label="Training loss")
plt.plot(baseline_history.history["val_loss"], marker="s", label="Validation loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Baseline CNN - Loss")
plt.legend()

plt.tight_layout()
plt.show()
```



In [25]:

```
# Evaluate on the test set

baseline_test_loss, baseline_test_acc = baseline_model.evaluate(test_pipeline, verbose=0)

print(f"Final training accuracy: {baseline_history.history['accuracy'][-1]:.4f}")
print(f"Final validation accuracy: {baseline_history.history['val_accuracy'][-1]:.4f}")
print(f"Test accuracy: {baseline_test_acc:.4f}")
print(f"Test loss: {baseline_test_loss:.4f}")
```

```
Final training accuracy: 0.9921
Final validation accuracy: 0.9912
Test accuracy: 0.9921
Test loss: 0.0297
```

In [26]:

```
# Predictions on the test set

y_prob = baseline_model.predict(test_pipeline, verbose=0)
y_pred = y_prob.argmax(axis=1)

misclassified = np.where(y_pred != y_test)[0]

print("Number of misclassified test images:", len(misclassified), "out of", len(y_test))
print(f"Test error rate: {len(misclassified) / len(y_test) * 100:.2f}%")
```

```
Number of misclassified test images: 79 out of 10000
Test error rate: 0.79%
```

In [27]:

```
# Confusion matrix

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False)
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Baseline CNN - Confusion Matrix (Test Set)")
plt.show()
```

Baseline CNN - Confusion Matrix (Test Set)

True Label	0	1	2	3	4	5	6	7	8	9
	977	0	1	0	0	0	1	1	0	0
	0	1135	0	0	0	0	0	0	0	0
	1	2	1019	0	1	0	0	9	0	0
	0	0	1	1006	0	2	0	0	1	0
	0	0	0	0	975	0	3	0	1	3
	0	0	0	8	0	882	1	0	1	0
	4	1	1	1	1	3	946	0	1	0
	0	2	3	0	0	0	0	1021	1	1
	3	0	2	0	0	0	0	2	966	1
	0	0	0	2	7	2	0	3	1	994
	0	1	2	3	4	5	6	7	8	9
Predicted Label										

In [28]:

```
# Classification report
```

```
print(classification_report(y_test, y_pred, digits=4))
```

	precision	recall	f1-score	support
0	0.9919	0.9969	0.9944	980
1	0.9956	1.0000	0.9978	1135
2	0.9922	0.9874	0.9898	1032
3	0.9892	0.9960	0.9926	1010
4	0.9909	0.9929	0.9919	982
5	0.9921	0.9888	0.9905	892
6	0.9947	0.9875	0.9911	958
7	0.9855	0.9932	0.9893	1028
8	0.9938	0.9918	0.9928	974
9	0.9950	0.9851	0.9900	1009
accuracy			0.9921	10000
macro avg	0.9921	0.9920	0.9920	10000
weighted avg	0.9921	0.9921	0.9921	10000

In [29]:

```
# Most common misclassifications

cm_errors = cm.copy()
np.fill_diagonal(cm_errors, 0)

error_pairs = []
for true_label in range(10):
    for pred_label in range(10):
        if cm_errors[true_label, pred_label] > 0:
            error_pairs.append((true_label, pred_label, cm_errors[true_label, pred_label]))

error_table = pd.DataFrame(error_pairs, columns=["True Digit", "Predicted Digit", "Count"])
error_table = error_table.sort_values("Count", ascending=False).head(5).reset_index(drop=True)

error_table
```

	True Digit	Predicted Digit	Count
0	2	7	9
1	5	3	8
2	9	4	7
3	6	0	4
4	7	2	3

In [30]:

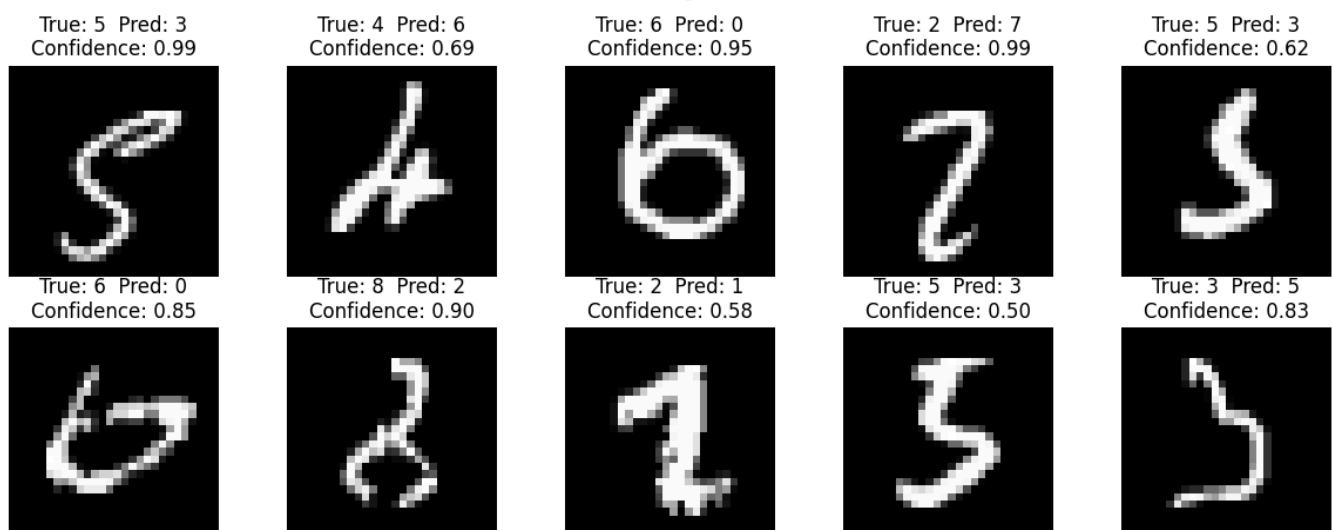
```
# Visualising misclassified images (predicted vs true)

plt.figure(figsize=(12, 5))

for i, index in enumerate(misclassified[:10]):
    plt.subplot(2, 5, i + 1)
    plt.imshow(X_test[index], cmap="gray")
    plt.title(f"True: {y_test[index]} Pred: {y_pred[index]}\nConfidence: {y_prob[index].max():.2f}")
    plt.axis("off")

plt.suptitle("Misclassified Test Images - Baseline CNN")
plt.tight_layout()
plt.show()
```

Misclassified Test Images - Baseline CNN



I trained the baseline CNN for 10 epochs using the Adam optimiser and sparse categorical cross-entropy loss. Validation accuracy was already 98.10% after the first epoch and finished at 99.12%, and the final test accuracy was 99.21% (test loss 0.0297), with 79 misclassified images out of 10,000. Validation accuracy was higher than training accuracy in the early epochs because dropout is only active during training, and by epoch 10 the train-validation gap was only 0.09 percentage points; however, validation loss was lowest at epoch 4 (0.0375) and then levelled off while training loss kept falling, which shows the model was starting to overfit slightly and that training for much longer would add little. The confusion matrix is strongly diagonal, and the most common errors were between digits with similar strokes: 2 predicted as 7 (9 times), 5 as 3 (8 times), 9 as 4 (7 times) and 6 as 0 (4 times), which matches digits 9, 2 and 6 having the lowest recall (98.51%, 98.74% and 98.75%) while every digit 1 was classified correctly. Many of the misclassified images are badly formed or ambiguous, such as a 2 written like a 7 or a 4 with an extra stroke, but the model was also very confident on some wrong answers (for example, a 5 predicted as 3 with 0.99 confidence), so the remaining errors come mainly from unusual handwriting rather than a lack of capacity, and data augmentation with small rotations and shifts would be a sensible next step.

TASK 5 — Controlled Architecture Change and Impact Comparison

Change Made

I changed **one** design parameter: the number of filters in the second convolution layer, from **64 to 32**. Everything else was kept the same (seed, data split, pipeline, optimiser, learning rate, batch size, dropout rates and number of epochs), so any difference in the results is caused by this change alone.

In [31]:

```
# Building the modified CNN (Conv2D-2: 64 -> 32 filters)

keras.utils.set_random_seed(SEED)
modified_model = build_cnn(conv2_filters=32, model_name="modified_cnn")

print("Modified CNN built.")
```

Modified CNN built.

In [32]:

```
# Modified model summary

modified_model.summary()
```

Model: "modified_cnn"

Layer (type)	Output Shape	Param #
conv1 (Conv2D)	(None, 28, 28, 32)	320
conv2 (Conv2D)	(None, 28, 28, 32)	9,248
maxpool (MaxPooling2D)	(None, 14, 14, 32)	0
flatten (Flatten)	(None, 6272)	0
dropout1 (Dropout)	(None, 6272)	0
dense1 (Dense)	(None, 128)	802,944
dropout2 (Dropout)	(None, 128)	0

output (Dense)	(None, 10)	1,290
----------------	------------	-------

Total params: 813,802 (3.10 MB)

Trainable params: 813,802 (3.10 MB)

Non-trainable params: 0 (0.00 B)

In [33]:

```
# Compile and train the modified CNN
# Rebuilt here for the same reason as the baseline: each run starts from fresh weights.
```

```
keras.utils.set_random_seed(SEED)
modified_model = build_cnn(conv2_filters=32, model_name="modified_cnn")
```

```
modified_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
keras.utils.set_random_seed(SEED)
modified_timer = EpochTimer()
```

```
modified_history = modified_model.fit(
    train_pipeline,
    validation_data=val_pipeline,
    epochs=EPOCHS,
    callbacks=[modified_timer],
    verbose=2
)
```

```
Epoch 1/10
844/844 - 8s - 10ms/step - accuracy: 0.9276 - loss: 0.2332 - val_accuracy: 0.9788 - val_loss: 0.0700
Epoch 2/10
844/844 - 6s - 7ms/step - accuracy: 0.9724 - loss: 0.0911 - val_accuracy: 0.9842 - val_loss: 0.0521
Epoch 3/10
844/844 - 10s - 12ms/step - accuracy: 0.9790 - loss: 0.0698 - val_accuracy: 0.9857 - val_loss: 0.0495
Epoch 4/10
844/844 - 6s - 8ms/step - accuracy: 0.9828 - loss: 0.0575 - val_accuracy: 0.9877 - val_loss: 0.0443
Epoch 5/10
844/844 - 6s - 7ms/step - accuracy: 0.9845 - loss: 0.0477 - val_accuracy: 0.9873 - val_loss: 0.0444
Epoch 6/10
844/844 - 6s - 7ms/step - accuracy: 0.9858 - loss: 0.0426 - val_accuracy: 0.9888 - val_loss: 0.0419
Epoch 7/10
844/844 - 6s - 7ms/step - accuracy: 0.9888 - loss: 0.0371 - val_accuracy: 0.9882 - val_loss: 0.0428
Epoch 8/10
844/844 - 6s - 8ms/step - accuracy: 0.9882 - loss: 0.0358 - val_accuracy: 0.9893 - val_loss: 0.0422
Epoch 9/10
844/844 - 6s - 7ms/step - accuracy: 0.9896 - loss: 0.0319 - val_accuracy: 0.9888 - val_loss: 0.0385
Epoch 10/10
844/844 - 6s - 7ms/step - accuracy: 0.9906 - loss: 0.0292 - val_accuracy: 0.9903 - val_loss: 0.0412
```

In [34]:

```
# Evaluate the modified CNN on the test set
```

```
modified_test_loss, modified_test_acc = modified_model.evaluate(test_pipeline, verbose=0)
```

```
print(f"Final training accuracy: {modified_history.history['accuracy'][-1]:.4f}")
print(f"Final validation accuracy: {modified_history.history['val_accuracy'][-1]:.4f}")
print(f"Test accuracy: {modified_test_acc:.4f}")
print(f"Test loss: {modified_test_loss:.4f}")
```

```
Final training accuracy: 0.9906
Final validation accuracy: 0.9903
Test accuracy: 0.9915
Test loss: 0.0299
```

In [35]:

```
# Comparison table (baseline vs modified)

def model_results(model, history, test_loss, test_acc, epoch_times):
    return {
        "Conv2D-2 filters": model.get_layer("conv2").filters,
        "Flatten size": model.get_layer("flatten").output.shape[-1],
        "Conv2D-2 params": model.get_layer("conv2").count_params(),
        "Dense-128 params": model.get_layer("dense1").count_params(),
        "Total params": model.count_params(),
        "Model size (MB)": round(model.count_params() * 4 / 1024**2, 2),
        "Final train accuracy": round(history.history["accuracy"][-1], 4),
        "Final val accuracy": round(history.history["val_accuracy"][-1], 4),
        "Train-val gap": round(history.history["accuracy"][-1] - history.history["val_accuracy"]
[-1], 4),
        "Test accuracy": round(test_acc, 4),
        "Test loss": round(test_loss, 4),
        "Avg seconds per epoch": round(float(np.mean(epoch_times[1:])), 2),
    }

baseline_results = model_results(baseline_model, baseline_history,
                                baseline_test_loss, baseline_test_acc, baseline_timer.times)
modified_results = model_results(modified_model, modified_history,
                                modified_test_loss, modified_test_acc, modified_timer.times)

comparison_table = pd.DataFrame({
    "Baseline (64 filters)": pd.Series(baseline_results, dtype=object),
    "Modified (32 filters)": pd.Series(modified_results, dtype=object),
})

comparison_table["Change"] = pd.Series(
    {k: round(modified_results[k] - baseline_results[k], 4) for k in baseline_results},
    dtype=object
)

comparison_table
```

	Baseline (64 filters)	Modified (32 filters)	Change
Conv2D-2 filters	64	32	-32
Flatten size	12544	6272	-6272
Conv2D-2 params	18496	9248	-9248
Dense-128 params	1605760	802944	-802816
Total params	1625866	813802	-812064
Model size (MB)	6.2	3.1	-3.1
Final train accuracy	0.9921	0.9906	-0.0015
Final val accuracy	0.9912	0.9903	-0.0009
Train-val gap	0.0009	0.0003	-0.0006
Test accuracy	0.9921	0.9915	-0.0006
Test loss	0.0297	0.0299	0.0002
Avg seconds per epoch	8.03	6.47	-1.56

In [36]:

```
# Summary of the changes

param_change = modified_model.count_params() - baseline_model.count_params()
param_change_pct = param_change / baseline_model.count_params() * 100
acc_change = (modified_test_acc - baseline_test_acc) * 100
time_change_pct = (np.mean(modified_timer.times[1:]) / np.mean(baseline_timer.times[1:]) - 1) * 100

print(f"Parameter change: {param_change:,} ({param_change_pct:.1f}%)")
print(f"Test accuracy change: {acc_change:+.2f} percentage points")
print(f"Training time per epoch change: {time_change_pct:+.1f}%")
```

```
Parameter change: -812,064 (-49.9%)
Test accuracy change: -0.06 percentage points
Training time per epoch change: -19.5%
```

In [37]:

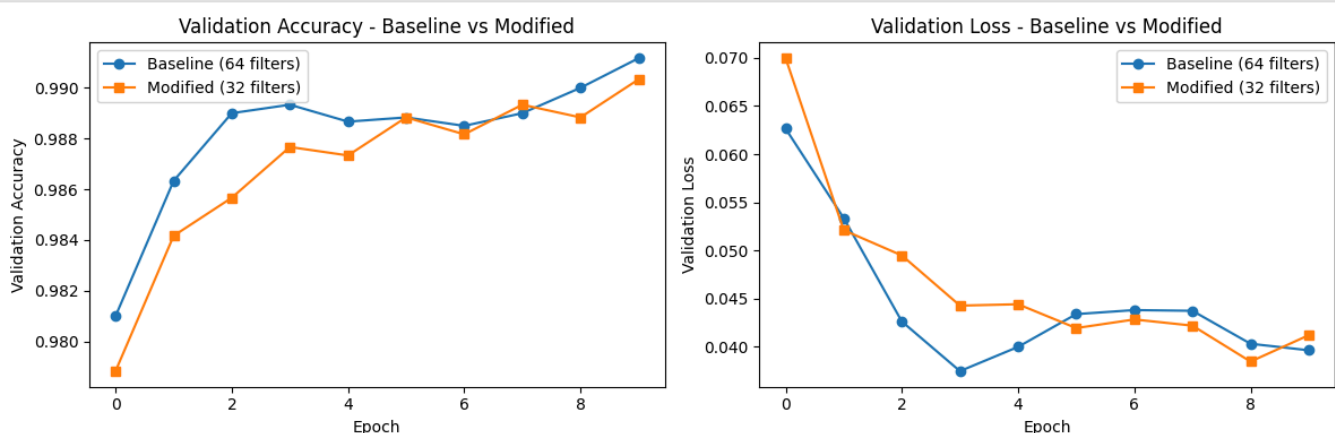
```
# Plot comparison of validation accuracy and loss

plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(baseline_history.history["val_accuracy"], marker="o", label="Baseline (64 filters)")
plt.plot(modified_history.history["val_accuracy"], marker="s", label="Modified (32 filters)")
plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.title("Validation Accuracy - Baseline vs Modified")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(baseline_history.history["val_loss"], marker="o", label="Baseline (64 filters)")
plt.plot(modified_history.history["val_loss"], marker="s", label="Modified (32 filters)")
plt.xlabel("Epoch")
plt.ylabel("Validation Loss")
plt.title("Validation Loss - Baseline vs Modified")
plt.legend()

plt.tight_layout()
plt.show()
```



In [38]:

```
# Extra check: repeating both models with different seeds
# (shows whether the accuracy difference is bigger than normal random variation)
# Runs automatically when a GPU is available; set RUN_EXTRA_SEEDS = True to force it on CPU.

RUN_EXTRA_SEEDS = len(gpus) > 0

if RUN_EXTRA_SEEDS:
    seed_results = []

    for seed in [1, 2, 3]:
        for filters in [64, 32]:
            keras.utils.set_random_seed(seed)
            model = build_cnn(conv2_filters=filters, model_name=f"cnn_{filters}_seed{seed}")
            model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001),
                          loss="sparse_categorical_crossentropy", metrics=["accuracy"])
            model.fit(train_pipeline, validation_data=val_pipeline, epochs=EPOCHS, verbose=0)
            _, acc = model.evaluate(test_pipeline, verbose=0)
            seed_results.append({"Seed": seed, "Conv2D-2 filters": filters, "Test accuracy": acc})
            print(f"Seed {seed} | {filters} filters | test accuracy {acc:.4f}")

    seed_table = pd.DataFrame(seed_results)
    display(seed_table.groupby("Conv2D-2 filters")["Test accuracy"].agg(["mean", "std"]).round(4))
else:
    print("Extra seed runs skipped (no GPU). Set RUN_EXTRA_SEEDS = True to run them on the CPU.")
```

```
Seed 1 | 64 filters | test accuracy 0.9909
Seed 1 | 32 filters | test accuracy 0.9910
Seed 2 | 64 filters | test accuracy 0.9911
Seed 2 | 32 filters | test accuracy 0.9919
Seed 3 | 64 filters | test accuracy 0.9914
Seed 3 | 32 filters | test accuracy 0.9927
```

	mean	std
Conv2D-2 filters		
32	0.9919	0.0009
64	0.9911	0.0003

Reducing the second convolution layer from 64 to 32 filters cut the Conv2D-2 parameters from 18,496 to 9,248, but the much bigger saving came from the dense layer. The flattened output fell from 12,544 to 6,272 values, so the Dense-128 layer dropped from 1,605,760 to 802,944 parameters. Overall, the model shrank from 1,625,866 to 813,802 parameters (a reduction of 812,064, or about 49.9%) and its size halved from 6.2 MB to 3.1 MB.

The test accuracy only changed from 99.21% to 99.15% (−0.06 percentage points, about 6 more errors out of 10,000). To check whether this difference was meaningful, I repeated both models with three further seeds: the 32-filter model averaged 99.19% (± 0.09 percentage points) and the 64-filter model 99.11% (± 0.03), so the smaller model was actually slightly better on average, and the difference seen with seed 42 is within normal random variation. This shows that MNIST digits are simple enough for 32 feature maps to capture the important strokes and edges, so the extra capacity of 64 filters is mostly redundant. The validation curves show the modified model learning a little more slowly in the first few epochs before catching up in the second half of training.

In terms of overfitting, both models generalised well, with a final train-validation gap of 0.09 percentage points for the baseline and 0.03 for the modified model, since the smaller model has less capacity to memorise the training data. For compute, halving the parameters also roughly halves the calculations, but on the T4 GPU each epoch only fell from 8.03 to 6.47 seconds

(19.5% faster). This is because such a small model does not fully use the GPU, so fixed costs such as moving batches to the GPU make up a large share of the time. For this task, the 32-filter model is the better trade-off because it gives practically the same accuracy with half the parameters, half the memory and less training time. However, on a harder dataset such as CIFAR-10, reducing capacity like this would be more likely to cause underfitting.

TASK 6 — Cloud / Deployment Readiness Reflection (Azure-aware)

Training this CNN is possible on a CPU, but in my runs one epoch took about two minutes on a Colab CPU compared with about 8 seconds on a T4 GPU, because convolutions run well in parallel. In Azure Machine Learning, I would therefore train on a GPU compute cluster that scales down to zero when idle, but serve predictions from a cheaper CPU-based endpoint, since classifying single images needs little compute. Because cloud compute is created and removed on demand, fixed seeds, pinned library versions in an Azure ML environment and versioned data are what make a run repeatable and auditable. One limitation is that MNIST contains clean, centred, greyscale digits, so the model may perform poorly on photographed, rotated or differently written digits, and it can be confidently wrong. If I deployed it as an Azure ML managed endpoint, I would monitor prediction confidence and data drift, log misclassified inputs for retraining, set budget alerts to control cost, and consider responsible-AI risks if predictions affected real decisions.

AI-Use Disclosure

I used Claude (Anthropic) as an AI assistant to help structure this notebook, draft code for the tf.data pipeline, CNN model, evaluation and comparison, and draft the explanatory text. I ran all the code myself, checked that the model summary matched the reference architecture, and verified and edited the explanations so that they reflect my own results and understanding.